

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T08:48:33.0196134Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-084833131.prof
Profil mémoire	memory-20260924-084833131.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	7.23 ms	7231574	59489	17

Détail hyperfine

Métrique	Valeur
Moyenne	5.9792 s
Écart-type	n/a
Médiane	5.9792 s
Temps utilisateur	28.3222 s
Temps système	1.2134 s
Min	5.9792 s
Max	5.9792 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	29.55s
Mémoire allouée (pprof)	100.01MB
Mémoire par opération	59489 B/op

Garbage collector Go

Cycles GC	69
Temps GC cumulé	68.880 ms

Plus longue pause	4.000 ms
Temps CPU GC cumulé	356.710 ms
Objectif mémoire maximal	95.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **29.55s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	24.17s	81.79%	27.50s	93.06%
jeu-de-la-vie/game.(*Board).index (inline)	2.61s	8.83%	2.64s	8.93%
jeu-de-la-vie/game.(*Board).Step	0.99s	3.35%	1.60s	5.41%
runtime.asyncPreempt	0.46s	1.56%	0.46s	1.56%
math/rand.(*Rand).Float64 (inline)	0.31s	1.05%	0.58s	1.96%
jeu-de-la-vie/game.squaredDistance (inline)	0.26s	0.88%	0.26s	0.88%
jeu-de-la-vie/game.(*Board).countInfectionCand	0.23s	0.78%	27.74s	93.87%
math/rand.(*rngSource).Uint64 (inline)	0.15s	0.51%	0.15s	0.51%

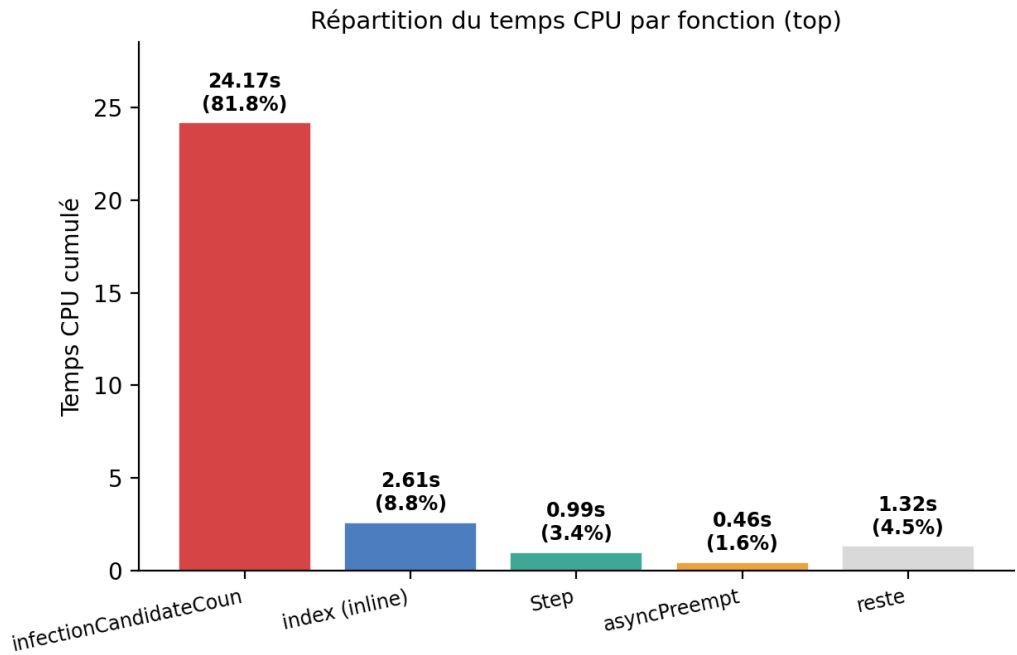


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

File: game.test.exe
 Build ID: C:\Users\salou\JeuDeLaVie\game.test.exe2026-09-24 10:48:14.6169794 +0200 CEST
 Type: cpu
 Time: 2026-09-24 10:48:33 CEST
 Duration: 4.48s, Total samples = 29.55s (660.24%)
 Showing nodes accounting for 29.30s, 99.15% of 29.55s total
 Dropped 82 nodes (cum <= 0.15s)
 See <https://git.io/JfYMW> for how to read the graph

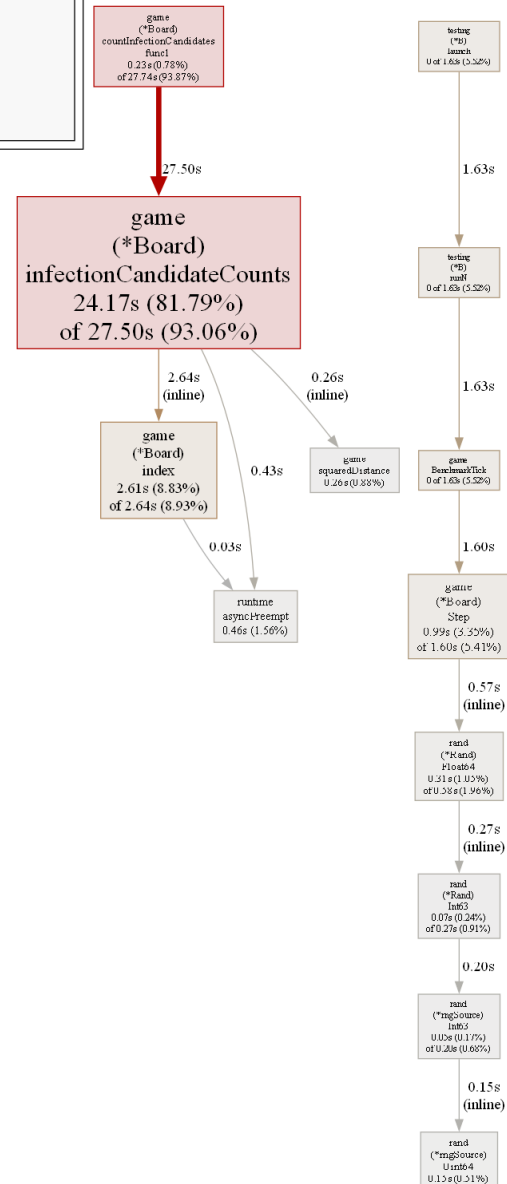


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).infectionCandidateCounts**, responsable de **81.8%** du temps CPU total (flat), pour **93.1%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

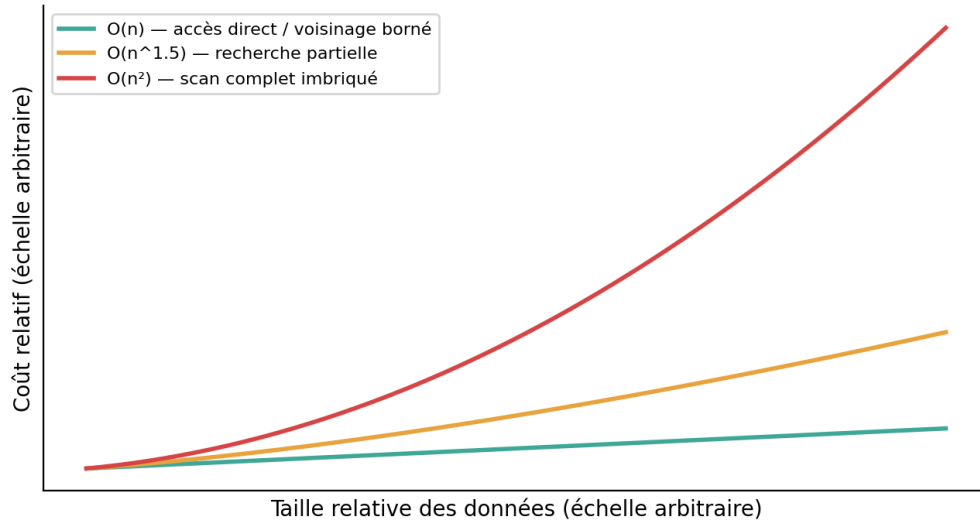


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **100.01MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).ensureScratch (inl	52.47MB	52.46%	52.47MB	52.46%
jeu-de-la-vie/game.NewBoard (inline)	19.33MB	19.33%	19.33MB	19.33%
jeu-de-la-vie/game.RandomBoard	16.57MB	16.57%	35.90MB	35.90%
runtime.mallocgc	5.01MB	5.01%	5.01MB	5.01%
runtime/pprof.StartCPUProfile	1.72MB	1.72%	1.72MB	1.72%
jeu-de-la-vie/game.(*Board).countInfectionCand	1MB	1.00%	1MB	1.00%
compress/flate.NewWriter (inline)	0.88MB	0.88%	1.41MB	1.41%
compress/flate.newDeflateFast (inline)	0.53MB	0.53%	0.53MB	0.53%

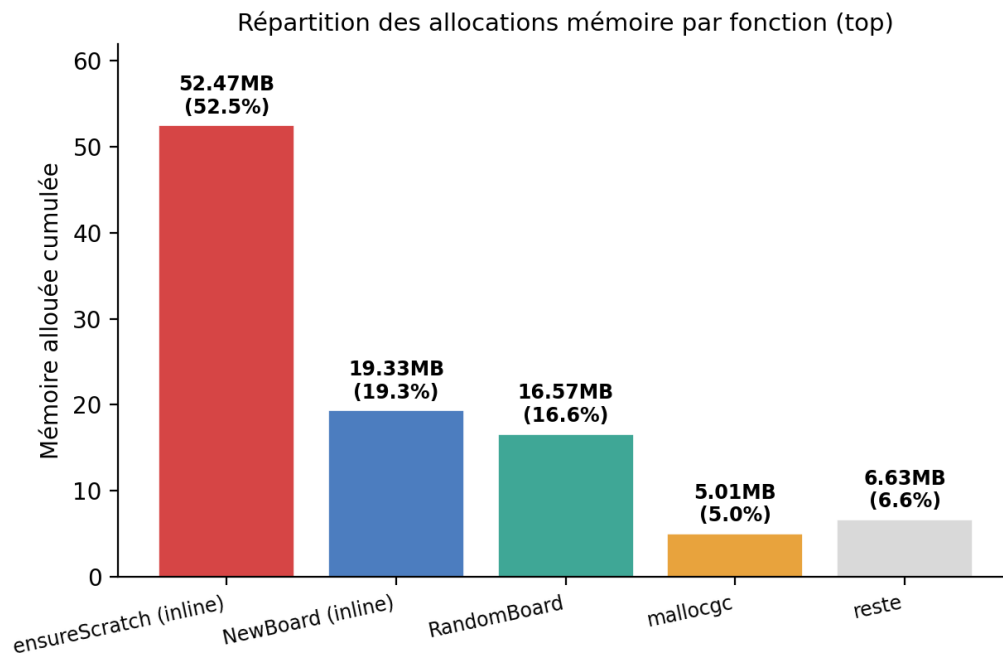


Figure — Répartition des allocations mémoire par fonction.

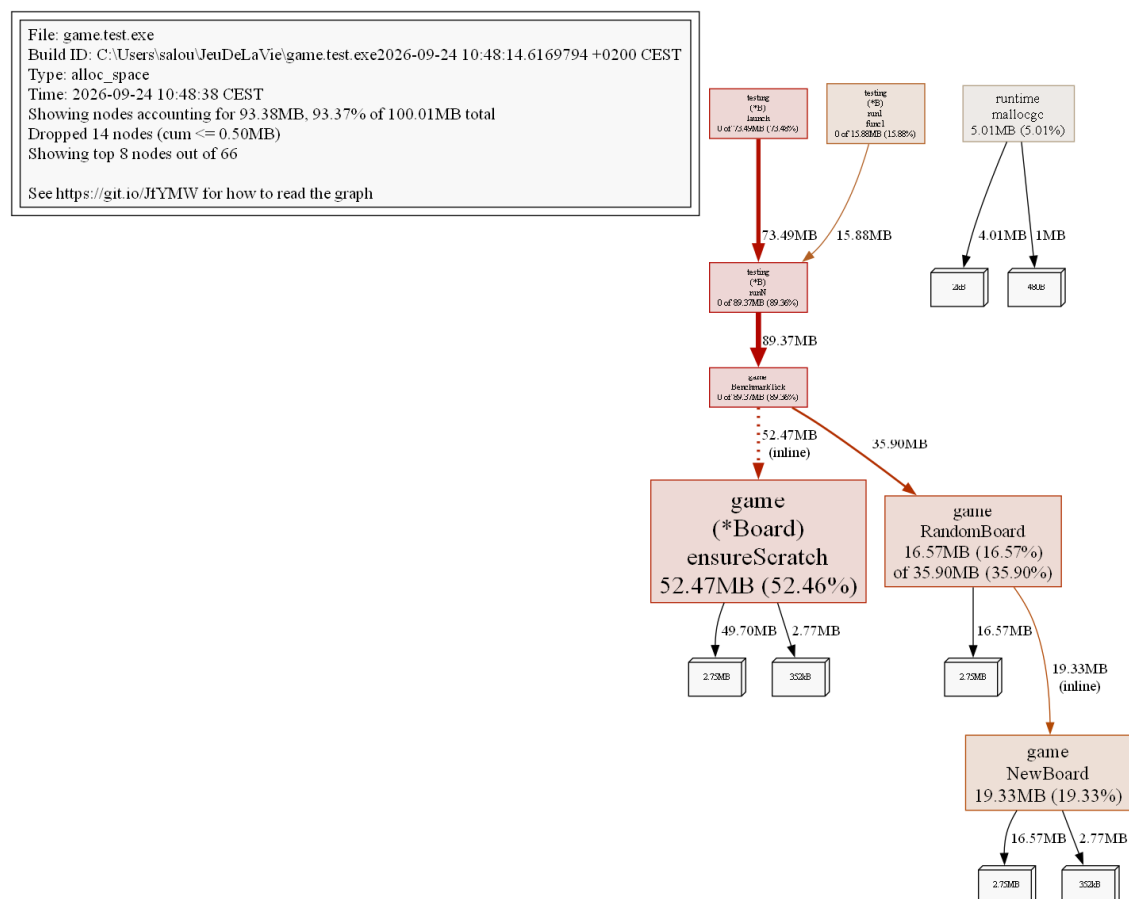


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 10:48:48 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.